

// RVASec 2026

Hacking Customized IDE

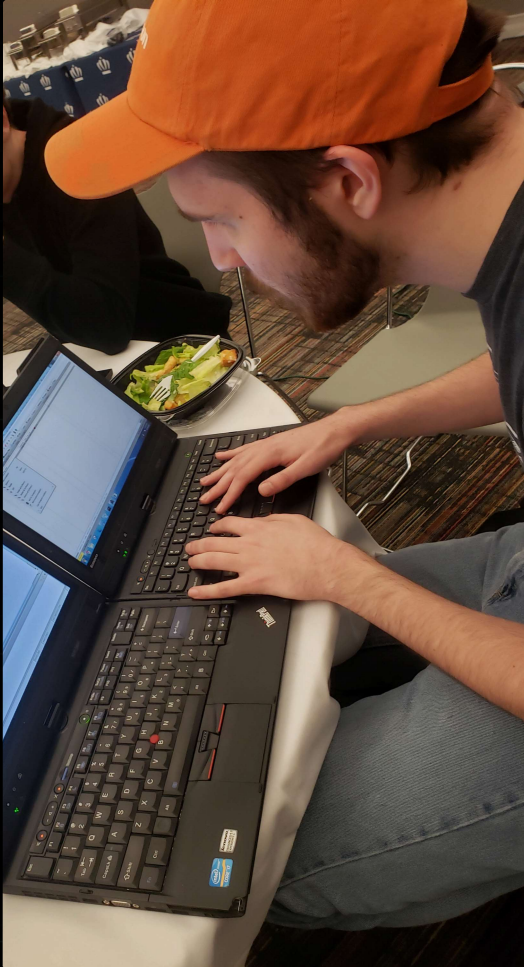
Distributions

Methodology behind six figures in bug bounties.

-- Nick Copi

```
// $ whoami
```

Nick Copi



- Full Time Bug Bounty Hunter
- ~~Recovering~~ Former AppSec Engineer at CarMax
- VCU Alumni, Former CyberSecurity Club President
- RVASec CTF Reigning Champion
- Thinkpad Dual Wielder
- nickcopi.com

More hours spent debugging JavaScript in the last year than in direct sunlight.

```
// Before we start
```

This is a methodology talk.

- No specific program, vendor, or live finding will be named.
- All examples are **generalized patterns**, abstracted across multiple targets.
- If you recognize a bug class here, that is the point. If you recognize a specific bug, that is a coincidence.

```
> Everything here applies across VS Code forks, custom JupyterLab plugins, and ground-up custom IDEs.
```

Why this is worth your time.

Attack economics

- developers are privileged users
- IDEs hold SCM tokens, cloud credentials, session cookies
- one developer compromise often equals a production compromise
- bounty programs pay well for developer-targeting bugs

The unclaimed surface

- mature web-app bug classes, freshly ported into IDEs
- custom distributions re-introduce classes that vendors already fixed upstream
- AI agents add a whole new sink layer
- few specialized hunters in this space

> Six figures came from the methodology in this deck.

// 01 / Framing

What even *is* a modern IDE?

Not a text editor. A **userland supervisor** that happens to show files.

- spawns shells and interpreters on demand
- holds SCM tokens, cloud creds, session cookies
- runs extensions with near-user privilege
- renders untrusted content as part of its job
- hosts AI agents that act on workspace input

The job description

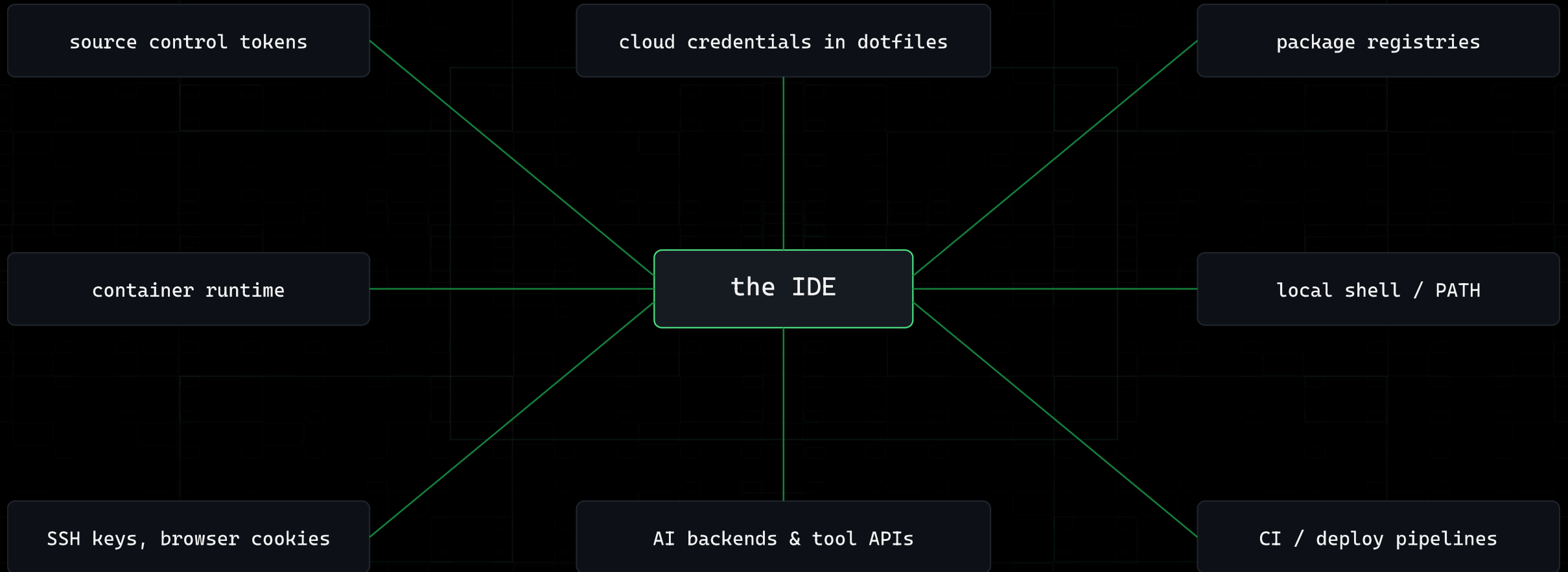
open(untrusted) → render() → suggest() → execute()

An IDE's core function is to consume attacker-controlled input and act on it. That's not a bug, that's the product.

> Every line of that job description is a bug class.

// 02 / Blast radius

What a compromised IDE reaches.



> Symlink-following file tools collapse the `~/` distance to zero – everything above is reachable from any workspace.

Three buckets that cover everything.

VS Code family

- forks (Cursor, VSCodium, etc.)
- curated extension profiles
- web-hosted (code-server, Gitpod, etc.)
- inherits upstream trust model
 - usually loosened

JupyterLab plugins

- custom frontend plugins
- custom server extensions
- kernel-trust assumptions inherited from upstream
- notebook format doubles as delivery mechanism

Custom IDEs

- ground-up web IDEs
- Electron-based native IDEs
- hybrid web + native backends
- no upstream security model to inherit – good and bad

> Each bucket reshapes the threat model: origin boundaries, sandboxing, filesystem access, credential storage.

Where it runs changes *what breaks*.

Local IDE

Electron app on your machine. Code, extensions, and any agent run under your user.

- `file://` URIs, URI-scheme handlers
- OS-level process isolation
- NTLM-on-Windows primitives
- full filesystem via your account
- cloud creds sitting in `~/` dotfiles

SaaS web IDE

Browser tab. Workspace lives on a vendor backend. Often multi-tenant.

- origin model *is* the security model
- multi-tenant same-origin = blast radius
- `postMessage` grids, service workers
- auth tokens in URL or cookies
- exec happens in a shared backend

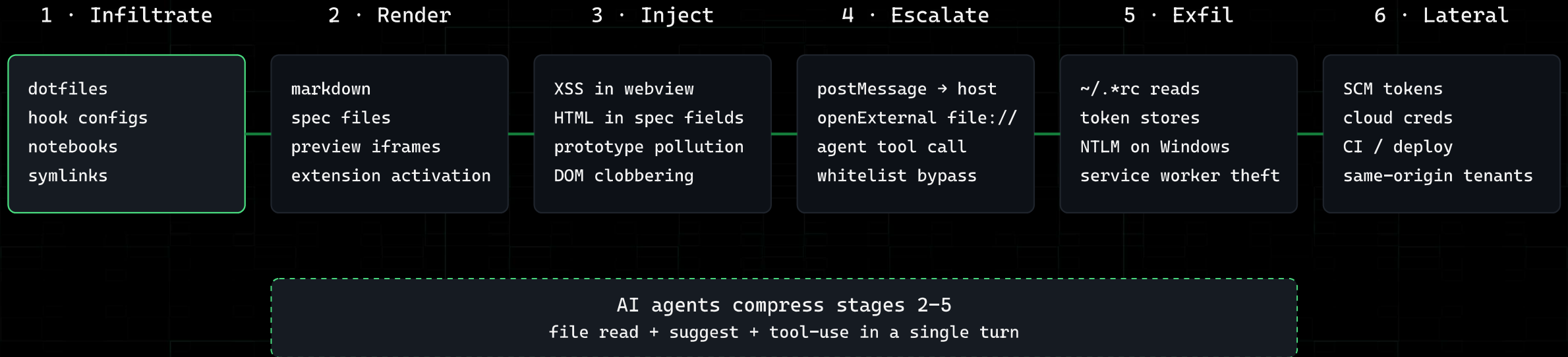
> Same bug class, different consequences. A rendered-markdown XSS in a local IDE is RCE under your account. In a SaaS web IDE it's a stepping stone toward every other tenant on that origin.

// Part 1

Threat modeling.

Build the map before you walk it.

The full chain.



Chain shapes in the wild.

- A. file → render → XSS → IPC → shell
- B. file present → auto-hook → shell (before trust)
- C. symlink → agent read → credential exfil via image / markdown src
- D. spec file → `javascript:` URL field → click-to-execute
- E. whitelisted-prefix agent call → glob bypass → metachar injection

These are gadgets and primitives – the moving parts of real chains. Chains are built from them.

Sources and sinks.

Sources – where input enters

- workspace files: dotfiles, project files, editor configs
- hook / task / launch / devcontainer definitions
- notebook cells, metadata, and rendered outputs
- markdown, HTML, visualization specs
- symlinks pointing outside the workspace
- custom URL-scheme and deeplink handlers
- clipboard, file paths, branch names – into prompts or shell commands; quoting differs cross-platform
- extension / plugin marketplace metadata

Sinks – where execution happens

- `postMessage` → extension host → terminal spawn
- `openExternal` with `file://` URIs
- `window.open` / `iframe src` with `javascript:`
- `innerHTML` / `document.write` from spec content
- shell execution via agent tools
- file writes into dotfiles, `tasks.json`, `shell-rc`
- service-worker registration

> Methodology: enumerate both lists for your target, then look for any path connecting left to right.

// 07 / Trust

Workspace trust is a *UI pattern*.

What it is

- a button the user clicks before "dangerous" features turn on
- a list of features gated behind that button

What it isn't

- filesystem isolation
- a sandbox
- a guarantee that *nothing* ran before the click

How custom distributions weaken it

- hooks / agents fire on folder-open, before the prompt
- "agentic mode" flagged as trusted regardless of workspace state
- implicit file reads (symlinks, preview, indexing) still hit the disk
- Windows UNC path reads leak NTLM hashes with no trust check
- extensions register activation events that preempt the gate

> The interesting window is **before the trust prompt is dismissed**.

Files that do more than sit there.

Config & hook files

- `.vscode/tasks.json` , `launch.json`
- `.devcontainer/*`
- custom agent config files
- MCP-style tool definitions
- lint / format / build hooks
- workspace-level editor config
 - processed on folder-open

Rendered content

- markdown with HTML
- notebook cells & outputs
- visualization specs (Vega, Plotly, etc.)
- SVG files opened as images
- help-doc / preview widgets

Filesystem tricks

- symlinks pointing to `~/`
- UNC paths on Windows
- workspace state files that restore widgets
- files whose *name* is the payload
- `.env` files auto-loaded on open

> Rule of thumb: if a file changes IDE behavior without being executed, it's a primitive.

From file on disk to code running.

- **Config-driven execution** – tasks, launch configs, Makefiles, pre-commit hooks parsed and run from workspace files.
- **Extension activation** – opening a file type wakes an extension that trusts the file it was activated for.
- **Preview pipelines** – rendering is executing. Any parser is a potential sink.
- **Whitelist bypass via glob** – `ls *` in a command allowlist matches `ls ; evil` once the shell expands it.
- **Path injection into shells** – `cd $path && tool` where `$path` is `/tmp; evil`.
- **Pre-trust execution** – hooks, agents, indexers that ran *before* the user was asked to trust the workspace.

Source → Bridge → Sink: attacker file → config parser → `child_process.spawn()`

When the UI lies to you.

Common in config-loading flows: the value the user *sees* during "preview" comes from one parser; the value that *runs* comes from another.

Where it shows up

- tool config manifests (MCP, agent configs, devcontainers)
- extension marketplace "about this extension" vs what installs
- notebook preview vs what `nbconvert` emits
- markdown preview vs markdown as saved
- command whitelist approval UI vs what the shell expands

Why it's a bug class

Any time two parsers handle the same bytes, the distance between them *is* the attack surface.

Diff both sides. If they diverge on any input, you have a primitive.

// 08 / Primitives

Symlink-follow before trust.

Agent, indexer, or search tool traverses a workspace, follows a symlink out of it, and reads something it has no business reading.

// plant

symlink to ~/.config/*rc

→

// trigger

open folder in IDE

→

// read

agent indexes contents pre-trust

One committed file in a repo. No additional interaction after folder-open. The trust prompt never fires – the read already happened.

// Part 2

Webviews.

Iframes with power they didn't deserve.

What a webview actually is.

An iframe with a private message channel back to the extension host.

- runs in a renderer process it usually shares with the editor
- has a CSP that the extension sets (or doesn't)
- can load "local resources" via a scheme the host brokers
- talks to the extension via `postMessage`, both directions

The security model in theory

- strict CSP blocks inline and remote script
- origin isolation via unique scheme
- extension host is the trust boundary
- messages are validated on both sides

In practice in custom builds

- CSP relaxed to make a library work
- origin isn't checked on incoming messages
- extension host exposes whatever's convenient
- "just use `innerHTML`, the CSP will catch it"

Webview implementation footguns.

- no origin check on incoming messages
- CSP with `unsafe-inline` "just for now"
- tokens and session secrets baked into the HTML
- message type dispatched as a command without an allowlist
- arbitrary string args forwarded to `executeCommand`
- HTML built by string concat from workspace file content

```
// vulnerable handler
window.addEventListener('message', e => {
  const { cmd, args } = e.data;
  vscode.postMessage({ cmd, args });
});

// the minimum
window.addEventListener('message', e => {
  if (e.source !== expectedSource) return;
  if (!ALLOWED.has(e.data.cmd)) return;
  const args = validate(e.data.cmd, e.data.args);
  if (!args) return;
  vscode.postMessage({ cmd: e.data.cmd, args });
});
```

If you're building webviews into your own IDE – **don't**. Fork VS Code and inherit its defaults.

Strict CSP helps, but it's a speed bump. `unsafe-inline`, wildcard sources, nonce reuse, and missing `frame-ancestors` are all things to look for – and CSP won't save you from a real `postMessage` routing bug anyway.

What's on the other side of that `postMessage` .

Any webview XSS that can forge a valid message to the host gets these, scoped to whatever the extension registered for.

Host-side capabilities

- spawn terminals / child processes
- read and write workspace files
- read editor configuration & global state
- `openExternal()` for URLs and `file://`
- register commands and keybindings
- call other extensions' APIs

What this means

- XSS + message-routing = RCE
- the trust boundary was the *message contract*, not the iframe
- strict origin + strict message schema = the real defense

// 10 / Escalation

Escalating XSS from a workspace file.

// source

workspace file

→

// render

markdown / custom
editor / preview

→

// xss

JS in webview
context

→

// escalate

postMessage → host
API

The ladder

1. workspace-controlled content reaches a webview renderer
2. incomplete HTML escaping, unsanitized spec field, or re-enabled tag leaves an injection point
3. XSS fires inside the webview's origin
4. XSS sends a `postMessage` the extension's host handler trusts
5. host handler calls a `terminal`, `file-write`, or `openExternal` API
6. code exec under the user, outside the webview sandbox

> Every rung here has been observed in the wild as a single-line mistake.

// Part 3

Web IDEs.

Nobody wants to run a thick app on their desktop.

postMessage, for the non-frontend folks.

A browser IPC mechanism. One window / iframe / worker sends a structured message to another; the receiver registers a listener.

Why it exists

- cross-origin frames can't read each other's DOM (same-origin policy)
- `postMessage` is the escape hatch – a message channel that works across origins
- it's opt-in on both ends: sender picks a target, receiver picks whether to listen

The API

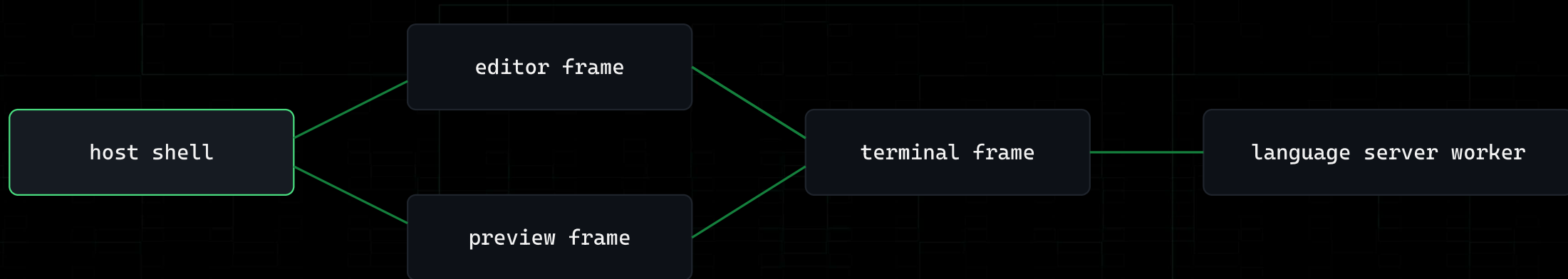
```
// sender
target.postMessage(data, targetOrigin);

// receiver
window.addEventListener('message', e => {
  e.origin    // who sent it
  e.source    // the sender's window
  e.data      // the payload
});
```

The threat model: the receiver decides what to trust. If the listener doesn't verify `e.origin` and `e.source`, anyone who can get a message to your window controls the logic it drives.

postMessage as a sprawling IPC fabric.

`window.postMessage(data, targetOrigin)` – cross-frame / cross-origin IPC. Receiver reads `event.data`, `event.origin`, `event.source`. The receiver decides what to trust.



- missing origin check – any opener can message you, including a popup from a phishing link
- missing source check – `e.source` ignored; any child frame counts
- command routing by string – no allowlist, messages dispatched as method names
- innerHTML from message payload – message content reaches a `div.innerHTML` unchanged
- cross-frame code exec – `window.opener.frames[0].eval(...)` patterns from a controlled popup

postMessage: a field guide.

```
// flag any of these in review
window.addEventListener('message', e => {
  // 1. no origin check
  // 2. no source check
  // 3. e.data used as command router directly
  // 4. e.data fields interpolated into innerHTML
  // 5. e.data.type dispatched without allowlist
});
```

What to verify

- is `e.origin` checked against a tight list?
- is `e.source` matched against a remembered window ref?
- is message `type` in a finite enum?
- is each payload schema-validated?

What to try as attacker

- send from a popup you opened with a crafted URL
- send from an iframe embedded in a page you control
- replay legitimate-looking types with swapped fields
- nest HTML in string fields that end up as `innerHTML`

Workers don't make the content safer.

Web IDEs push work off the main thread:

- language servers (parse, analyze)
- linters, formatters, build preflight
- notebook kernel adapters, protocol shims
- search / indexing over workspace files

The worker sees the same untrusted content the main thread does. It just sees it in a different realm.

What goes wrong

- worker `onmessage` trusts message type without validation
- worker reads workspace file paths that control behavior (e.g. load a grammar file)
- worker sends structured objects back that the main thread hands to `innerHTML`
- worker imports scripts by URL where the URL is workspace-derived
- transferable buffers from untrusted content parsed as code

> The worker boundary is a process boundary, not a trust boundary.

Rendering untrusted HTML in a web IDE.

The IDE's job is to preview content the user is editing. That content is attacker-controlled by definition.

Sandboxing choices

- `iframe srcdoc` – inherits parent origin; only isolated if you add `sandbox` without `allow-same-origin`
- `blob: URL` – inherits the creator's origin
- `same-origin iframe` – *not* a sandbox
- `sandbox="allow-scripts allow-same-origin"` – **defeats itself**; child reaches into parent functions

Where they fail

- CSP left off because the framework handles it (it doesn't)
- DOM clobbering via `<form>` / `<input>` in preview
- prototype pollution via merged workspace config
- incomplete entity encoding re-enables dangerous tags
- renderer loads assets from workspace-controlled URLs

// Part 4

Extensions & plugins.

Custom extensions shipped *with* the IDE – your code, your problem.

VS Code extension footguns.

- **shell from workspace strings** – paths, branch names, file names concatenated into commands
- **"cd \$path && ..."** – classic, still everywhere
- **glob whitelists** – `ls *` matches `ls; evil` after expansion
- **overly broad URI handlers** – `vscode://ext/open?url=...` accepting any URL
- **webview** → `executeCommand` – messages routed as method names
- **extension host ≠ trust boundary** – workspace code gets there just fine
- **auto-loading** `.env` from cwd on folder open

```
// don't
exec(`cd ${workspacePath} && npm test`);

// don't
if (ALLOWED_PREFIXES.some(p => cmd.startsWith(p)))
  shell(cmd);

// do
spawn('npm', ['test'], { cwd: workspacePath });
```

> Every one of these lines has shown up in extension code that shipped.

Extensions activate on *file presence*.

Opening a folder is enough to run arbitrary extension code, even if you never open a single file.

Common activation triggers

- `onLanguage:*` – a file of a given extension exists
- `workspaceContains:*` – a pattern matches any file
- `onStartupFinished` – every open fires it
- `onView:*` – side panel in the activity bar
- `onUri:*` – deeplink handler

Why this matters

- attacker controls what files are in the workspace
- attacker picks which extensions get activated
- extension code is trusted by default, workspace isn't
- the trust prompt gates the wrong boundary

> The set of installed extensions is the effective attack surface, whether the user thinks about it that way or not.

The extension marketplace is an attack surface.

- **typo-squatting** – `prettier` vs `pretier`, `eslint` vs `es-lint`
- **namespace collisions** – custom distributions bundle their own marketplaces that may shadow the real one
- **update-channel abuse** – extension updates don't re-prompt for trust
- **metadata vs binary divergence** – reviewed manifest, published something else
- **recommended-extension files** – `.vscode/extensions.json` auto-suggests installs
- **custom registry URLs** – config file points at an attacker-controlled registry

> For a custom IDE distribution, the marketplace story is often the weakest link in the whole chain.

Custom JupyterLab plugins.

The format *is* the delivery

- notebooks carry code, outputs, widgets, HTML, and metadata
- workspace state files restore UI and iframe src on open
- Vega / Vega-Lite specs ship with URL fields that render as links or frames
- nbconvert templates process notebook input into HTML / slides / scripts
- SVG images opened in a new tab land on a real origin

Plugin-side failures

- frontend plugin renders notebook output as HTML without sanitization
- server extension handler trusts path params from the frontend (traversal)
- server-extension routes ship with plain Python vulns – shell injection, traversal – that standard SAST often misses
- panel previews rendered via authed endpoints – CSRF reaches the kernel
- widget / help-doc frames take attacker-controlled `src`
- auto-run kernel code on notebook open bypasses trust
- custom help and preview widgets render untrusted HTML as trusted

> Frontend plugins and server extensions have very different privilege levels. Find where they trust each other.

// Part 5

AI agents.

The killchain accelerator.

AI agents compress the killchain.

One agent turn can read a file, interpret instructions embedded in it, call a tool, write to disk, and open a URL – with one user approval.

The five stages in one turn

- 1 · **infiltrate** – attacker-authored file in the workspace
- 2 · **render** – file fed to the model as context
- 3 · **inject** – file contains prompt instructions the model follows
- 4 · **escalate** – agent calls a tool under its own permissions
- 5 · **exfil** – tool response or subsequent write leaks data

Agent-specific primitives

- **symlink-following search tools** – `~/` is one link away
- **command whitelist via glob** – shell metachars pass the check
- **path / branch name prompt injection** – content reaches the model via operational metadata
- **hook config auto-executed** – before any trust prompt
- **agent-approved writes** to dotfiles, shell-rc, task configs

Trust fatigue: users click approve because the agent asked nicely. Per-tool confirmation is the only thing between an attacker-controlled string and shell execution.

Prompt injection: same pattern, new context.

Attacker data flows into a privileged execution context and is interpreted there.

Classic XSS

- attacker string reaches a page
- browser parses it as HTML / JS
- executes in the page's origin
- has access to DOM, cookies, storage

Prompt injection

- attacker string reaches a model
- model interprets it as instructions
- executes in the agent's tool scope
- has access to shell, files, terminals

The agent is the parser. Every field that becomes prompt context needs to be treated like untrusted HTML was in 2010.

// Part 6

Putting it together.

Three chains, one methodology.

// 15 / Putting it together

Chain: file preview to RCE.

A real chain, generalized. All in an untrusted workspace – no trust prompt dismissed.

1. **Plant.** Attacker ships a repo. It contains a file whose *path* embeds HTML-breaking characters, plus a companion binary that the OS will execute via its default handler (`.jar`, `.hta`, `.desktop`, depending on platform).
2. **Preview.** Victim clicks the file. The IDE opens a preview webview and interpolates the file's path into the page template unsanitized. The payload is crafted so an unclosed attribute consumes the CSP `<meta>` tag before the parser reaches it.
3. **XSS.** Attacker JavaScript now runs inside the webview's privileged origin.
4. **Escalate.** The webview's `postMessage` handler on the extension side accepts an `openExternal` action without allowlisting the URL scheme.
5. **RCE.** The XSS sends a message asking to `openExternal` a `file://` URL pointing at the companion binary in the same repo. The OS's default handler runs it.

Six bug classes, one click: path → HTML, CSP bypass, webview XSS, IPC without allowlist, `openExternal` as a sink, trust model gap. Each line lives in a different component.

// Part 7

Defense.

How to stop shipping these.

If you build an IDE.

Number one: **don't**. But if you do...

- defer *all* workspace-driven execution until trust is granted, including indexing, hooks, and agent config
- every `postMessage` handler: strict origin, strict source, enumerated message types, validated payloads
- never build a shell command by string concat from workspace content – use `spawn` with an argv array
- don't pair a command allowlist with a shell interpreter; one or the other
- reject `allow-same-origin` + `allow-scripts` on any untrusted iframe
- scope AI agent tools behind explicit per-invocation confirmation, especially for filesystem and shell
- assume the filesystem is hostile
- multi-tenant means multi-origin; do not share an origin across tenants

If you ship your own distribution.

By default VS Code extensions don't activate against untrusted workspaces – they have to opt in via `capabilities.untrustedWorkspaces` in `package.json`. If your extension does opt in (often reasonable: read-only helpers, language servers), the workspace is hostile and every path, message, and string crossing your boundary needs to be treated that way.

- **scope activation events tightly** – `onLanguage` / `onView` beats `onStartupFinished`; fewer entry points is less attack surface
- **treat the webview boundary as hostile** – same `postMessage` discipline you'd apply to a real RPC endpoint
- **schema-validate every message** – finite type enum, validated payloads, reject the rest
- **never forward workspace-sourced strings** to `executeCommand`, `openExternal`, or a shell – use structured APIs with argv arrays
- **don't pre-register unnecessary URI handlers** you haven't threat-modeled – a `vscode://your-ext/open` with a URL parameter is dangerous attack surface
- **keep agents gated** – per-tool, per-invocation confirmation for anything that touches filesystem, shell, or network
- **test with hostile workspaces** – your extension runs on code you didn't write; make sure the red-team build passes

IDEs are the choke point.

Source control. Cloud
credentials. Container runtimes.
Package registries. AI backends.

Every system a developer
touches, the IDE touches.
Compromise the IDE and you
inherit all of them.

That's what makes it worth
attacking – and worth taking
seriously on the other side.

Thanks

Nick Copi

questions welcome.